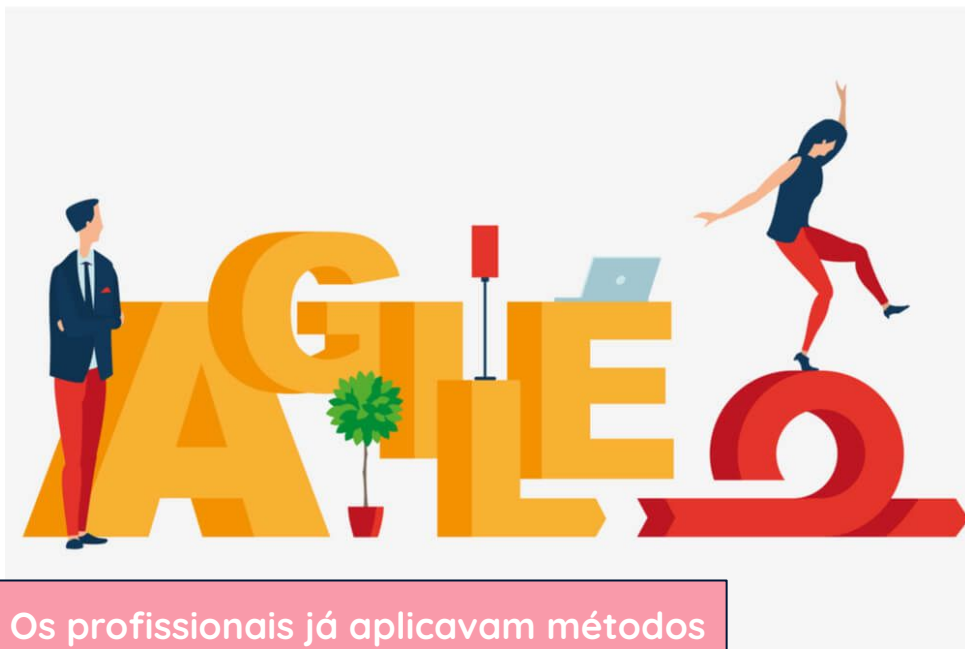


# METODOLOGIA DE DESENVOLVIMENTO ÁGIL

Vamos descobrir o que  
é desenvolvimento ágil





Os profissionais já aplicavam métodos ágeis como XP, Scrum, FDD, DSDM, etc.

Métodos diferentes mas com os mesmos fundamentos

# DESENVOLVIMENTO ÁGIL

2001: Kent Beck + 16 renomados autores, desenvolvedores e consultores da área de software

**Aliança dos Ágeis: *Agile Alliance***

**MANIFESTO PARA O  
DESENVOLVIMENTO ÁGIL DE  
SOFTWARE**

"Ao desenvolver e ajudar outros a desenvolver software, desvendamos formas melhores de desenvolvimento. Por meio deste trabalho passamos a valorizar:

1. **Indivíduos e interações** acima de processos e ferramentas
2. **Software operacional** acima de documentação completa
3. **Colaboração dos clientes** acima de negociação contratual
4. **Respostas a mudanças** acima de seguir um plano

Ou seja, embora haja valor nos itens à direita, valorizaremos os da esquerda mais ainda."

—ALIANÇA ÁGIL

# DESENVOLVIMENTO ÁGIL



## O QUE É

Princípio de desenvolvimento que prioriza entrega mais do que análise e projeto (embora essas atividades não sejam desencorajadas)

Também prioriza a comunicação ativa e contínua entre desenvolvedores e clientes



## QUAIS AS ETAPAS

As atividades básicas (comunicação, planejamento, modelagem, construção e entrega) se mantêm

Tarefas mínimas, com foco na entrega



## QUEM REALIZA

Engenheiros de software e outros envolvidos (gerentes, clientes, usuários)

**Equipe ágil**



## QUAL É O ARTEFATO

Incremento de software é o artefato mais importante de ser entregue adequadamente, na data combinada



## POR QUE É IMPORTANTE

O ambiente moderno de sistemas e dos produtos da área é acelerado e está em constante mudança. A engenharia ágil é uma alternativa muito interessante



## COMO GARANTIR QUALIDADE?

A equipe de software concorda que o processo funciona e a equipe produz incrementos de software passíveis de entrega e que satisfaçam o cliente



# O QUE É UM MANIFESTO?



# MANIFESTO

Normalmente é associado a um movimento político emergente

**Ataca a velha guarda e sugere uma mudança revolucionária**

Espera-se que essa mudança seja para melhor

O Manifesto Ágil trata da mesma coisa

As ideias já existiam há algum tempo, mas não haviam sido aplicadas ainda

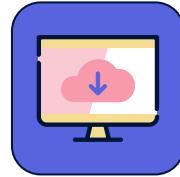
Está associado ao setor tecnológico, mas pode ser aplicado a outros setores

# PRINCÍPIOS POR TRÁS DO MANIFESTO ÁGIL



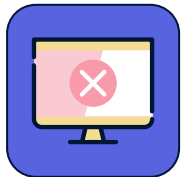
## PRIORIDADE

Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.



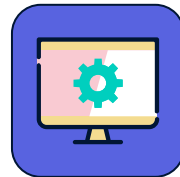
## ENTREGAS FREQUENTES

Entregar frequentemente software funcionando, de poucas semanas a poucos meses, com preferência à menor escala de tempo.



## MUDANÇAS NO SISTEMA

Mudanças nos requisitos são bem-vindas, mesmo tardiamente no desenvolvimento. Processos ágeis tiram vantagem das mudanças visando vantagem competitiva para o cliente.



## TRABALHO EM CONJUNTO

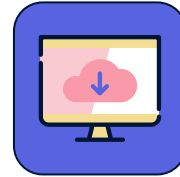
Pessoas de negócio e desenvolvedores devem trabalhar diariamente em conjunto por todo o projeto.

# PRINCÍPIOS POR TRÁS DO MANIFESTO ÁGIL



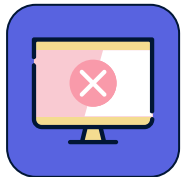
## INDIVÍDUOS MOTIVADOS

Construa projetos em torno de indivíduos motivados. Dê a eles o ambiente e o suporte necessário e confie neles para fazer o trabalho.



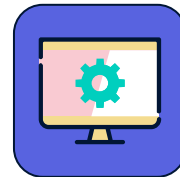
## MEDIDA DE PROGRESSO

Software funcionando é a medida primária de progresso.



## CONVERSAS FACE A FACE

O método mais eficiente e eficaz de transmitir informações para e entre uma equipe de desenvolvimento é através de conversa face a face.



## DESENVOLVIMENTO SUSTENTÁVEL

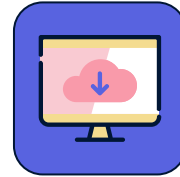
Os processos ágeis promovem desenvolvimento sustentável. Os patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.

# PRINCÍPIOS POR TRÁS DO MANIFESTO ÁGIL



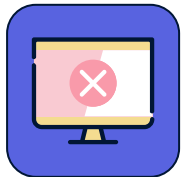
## AGILIDADE

Contínua atenção à excelência técnica e bom design aumenta a agilidade.



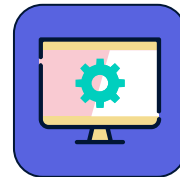
## EQUIPES AUTO-ORGANIZÁVEIS

As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis.



## SIMPLICIDADE

Simplicidade -- a arte de maximizar a quantidade de trabalho não realizado -- é essencial.



## EQUIPE EFICAZ

Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz e então refina e ajusta seu comportamento de acordo.



# QUANDO IREMOS APLICAR OS PRINCÍPIOS DO DESENVOLVIMENTO ÁGIL?

# PROBLEMAS ENCONTRADOS NA REALIDADE

Como o sistema vai evoluir com o tempo?

As necessidades do usuário mudaram?

É possível definir todos os requisitos no início do projeto?

**Agilidade:** dar uma resposta a um ambiente de negócios fluido





# DESENVOLVIMENTO ÁGIL

## É O FIM DA

# ENGENHARIA DE SOFTWARE?

Desenvolvimento ágil não significa que nenhum documento é criado;  
Significa que apenas os documentos que vão ser consultados mais adiante no processo de desenvolvimento são criados.

01.

O QUE É

AGILIDADE

Agilidade não é uma desculpa para  
abreviar soluções

Processo é um requisito, e disciplina é  
essencial



Atualmente, agilidade se tornou a palavra da moda quando se descreve um processo de software moderno. Todo mundo é ágil. Uma equipe ágil é aquela rápida e capaz de responder de modo adequado às mudanças.

Mudança tem tudo a ver com desenvolvimento de software.

Mudança no software que está sendo criado, nos membros da equipe, oriunda das novas tecnologias, mudanças de todos os tipos que poderão ter um impacto no produto que está em construção ou no projeto que cria o produto.

Suporte à mudança deve ser incorporado a tudo o que fazemos em software.

Uma equipe ágil reconhece que o software é desenvolvido por indivíduos trabalhando em equipes e que as habilidades dessas pessoas, suas capacidades em colaborar estão no cerne do sucesso do projeto.

— **IVAR JACOBSON (2002)**

# 4 VALORES DO MANIFESTO ÁGIL

## INDIVÍDUOS E INTERAÇÕES

A interação entre pessoas pode resolver problemas crônicos de comunicação

Processos e ferramentas são úteis, mas devem ser simples e úteis

## PROCESSOS E FERRAMENTAS

## SOFTWARE EM FUNCIONAMENTO

Clientes querem **resultado**: software funcionando

Documentação é importante, mas somente se for necessária e agregue valor

## DOCUMENTAÇÃO ABRANGENTE

## COLABORAÇÃO COM O CLIENTE

Focar na tomada de decisão em conjunto, inclusão do cliente, trabalho em equipe

Todos trabalham como um só em busca de um objetivo

## NEGOCIAÇÃO DE CONTRATOS

# 4 VALORES DO MANIFESTO ÁGIL

## RESPONDER A MUDANÇAS

Desenvolver software é um ambiente de alta incerteza

Não podemos confiar em planos enormes e cheios de premissas

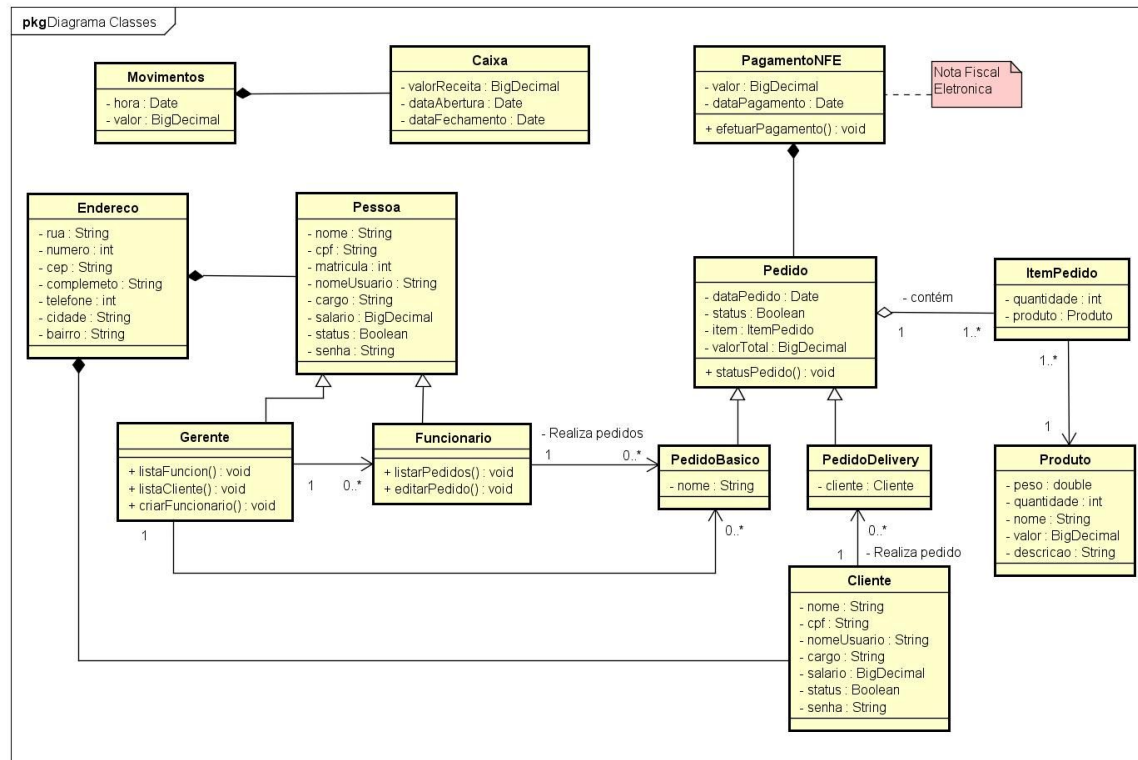
**O que acontece quando essas premissas não são garantidas?**

Devemos aprender com as informações e feedbacks e adaptar o plano a todo momento

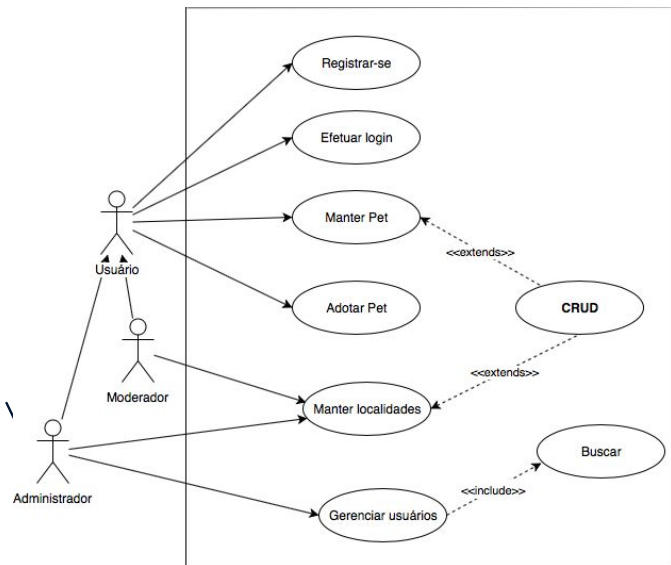
## SEGUIR UM PLANO



# DOCUMENTAÇÃO ABRANGENTE



# DOCUMENTAÇÃO ABRANGENTE



## VENDER CDS - CASO DE USO

### NOME

Vender CDs

### DESCRIÇÃO SUCINTA

Atendente vende um ou mais CDs a um usuário.

### ATORES

1. Atendente

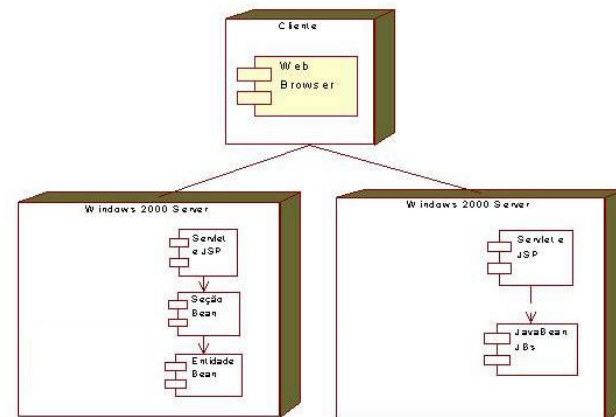
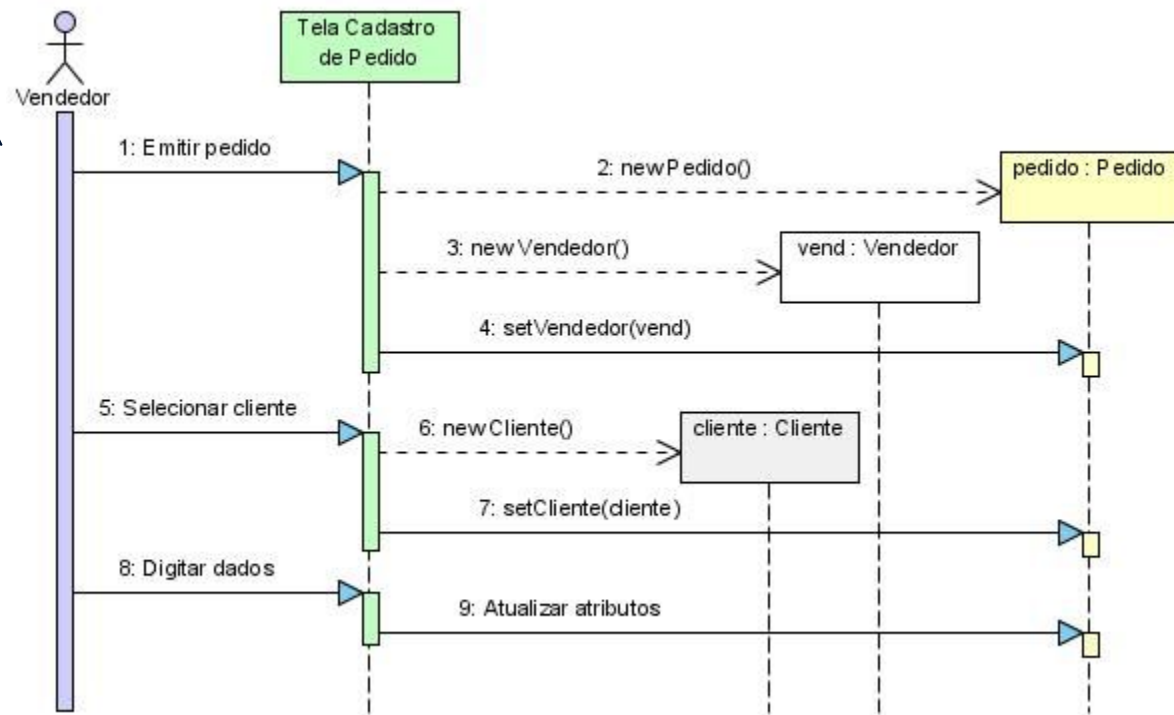
### PRÉ-CONDIÇÕES

1. Ter executado o caso de uso "CDU000\_Validar Senha"

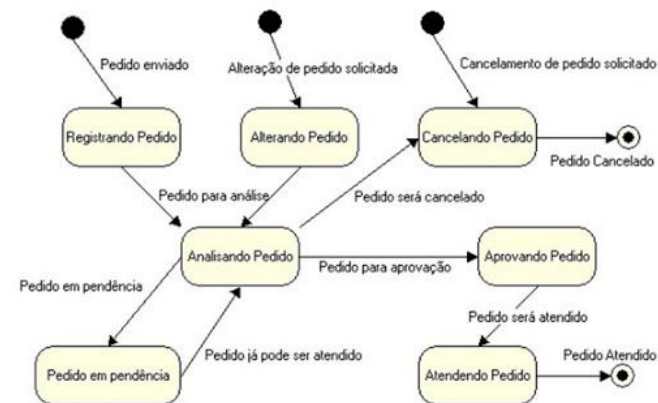
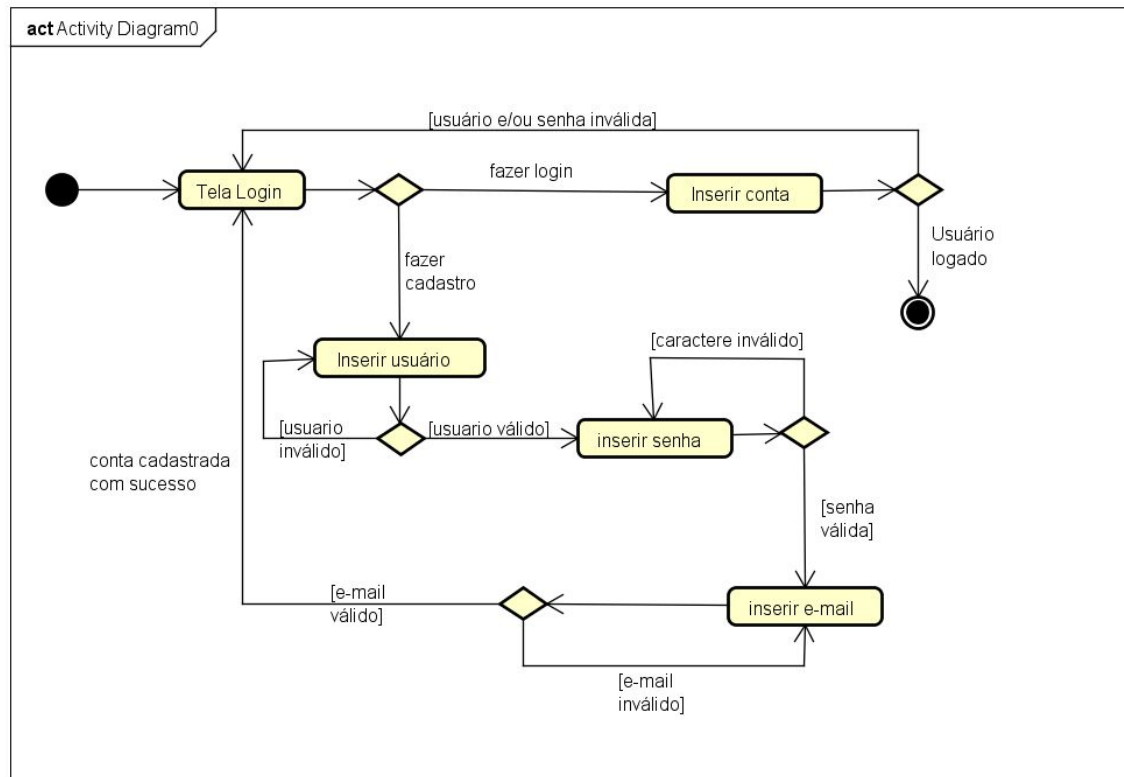
### FLUXO BÁSICO

1. O Atendente seleciona a opção "Vender CDs".
2. O Sistema exibe a lista de CDs.
3. O Atendente seleciona os CDs, informando as respectivas quantidades.
4. O Sistema exibe a lista de clientes.
5. O Atendente seleciona o cliente.
6. O Atendente seleciona a opção "Vender".
7. O Sistema exibe as informações da venda: CDs, quantidades e o cliente.
8. O Atendente confirma as informações da venda.
9. O Sistema efetua a venda, verificando a regra RN1.
  - 9.1. O Atendente seleciona o tipo de venda "A Prazo" ou "À Vista"
  - 9.2. O Sistema deve executar o caso de uso "CDU001a\_Vender CDs a prazo" ou o caso de uso "CDU001b\_Vender CDs à vista", de acordo com a opção selecionada pelo atendente no passo anterior.
  - 9.3. O Sistema atualiza o estoque de acordo com a regra RN2.
10. O Sistema emite a Nota Fiscal conforme ED1.
11. O caso de uso é encerrado.

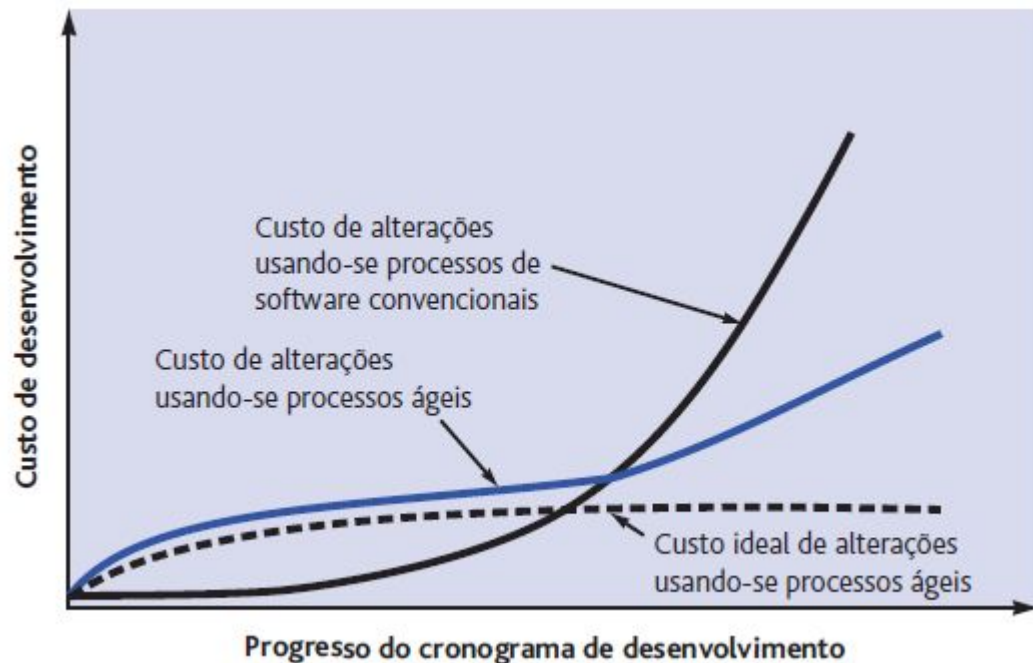
# DOCUMENTAÇÃO ABRANGENTE



# DOCUMENTAÇÃO ABRANGENTE



# AGILIDADE E O CUSTO DAS MUDANÇAS



Que pontos do desenvolvimento ágil achata o custo da curva de mudanças?

Entregas incrementais  
Testes contínuos  
Programação em pares  
Entre outras medidas



# A POLÍTICA DO DESENVOLVIMENTO ÁGIL

JIM HIGHSMITH (2002)



## A FAVOR ("AGILISTAS")

"Os metodologistas tradicionais preferem produzir uma documentação sem falhas em vez de um sistema que funcione e atenda às necessidades do negócio"



## CONTRA

"Os metodologistas de pouco peso são pretensiosos e vão acabar tendo uma grande surpresa ao tentarem transformar seus brinquedinhos em software de porte empresarial"

02.

EXTREME

PROGRAMMING

Conhecido como XP  
Programação Extrema



# EXTREME PROGRAMMING (XP)

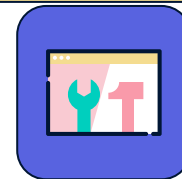
## APLICABILIDADE

- Qualquer tamanho de equipe
- Qualquer tamanho de projeto

## PONTOS IMPORTANTES DA METODOLOGIA

- A simplicidade é necessária desde a forma como se levanta requisitos até a codificação e os testes da solução desenvolvida;
- A comunicação é obrigatória para que não haja lacunas;
- O feedback é a prática fundamental, desde responder e-mails, telefonemas bips e demais meios;
- E a coragem para saber dizer NÃO quando necessário, ou então para dizer que o projeto vai demorar além do estimado, pois os novos requisitos precisam ser codificados ou o código já em funcionamento precisa ser refatorado.

# EXTREME PROGRAMMING (XP)



## METODOLOGIA

Orientada a objetos



## ATIVIDADES METODOLÓGICAS

Planejamento, projeto,  
codificação e testes

# XP: PLANEJAMENTO



## INÍCIO

- Levantamento de requisitos
- Capacitar os membros da equipe a entender o ambiente de negócio

## OUVIR

## HISTÓRIAS DE USUÁRIO

Descreve o resultado, as características e as funcionalidades solicitadas para o software

Armazenada em uma ficha

## PRIORIDADE

O cliente atribui um valor (prioridade) a cada ficha, baseando-se em seu valor de negócio

## CUSTO

Os membros da equipe atribuem um custo a cada ficha

Medido em semanas de desenvolvimento

## DIVISÃO DE FICHAS

- Se o custo for maior do que 3 (três) semanas de desenvolvimento, a ficha deve ser dividida em fichas menores
- Prioridade e custo devem ser redefinidos

## ESCOLHA DE HISTÓRIAS PARA O NOVO INCREMENTO

Feito por cliente e desenvolvedores em conjunto

Define as histórias e as datas de entrega

## ORDENAÇÃO DAS HISTÓRIAS

1. Todas são implementadas imediatamente
2. As histórias com maior valor são implementadas primeiro
3. As histórias com maior risco são implementadas



# XP: PLANEJAMENTO

## VELOCIDADE

Depois da entrega é calculada a velocidade do projeto

**Número de histórias implementadas durante a primeira versão**

## ALTERAÇÃO DO ESCOPO

Adição de histórias

Mudar valores

Dividir histórias

Remover histórias

**Pode acontecer a qualquer momento**

## ANÁLISE DA VELOCIDADE DO PROJETO

1. Ajuda a estimar as datas de entrega e o cronograma para versões seguintes
2. Determina se foi assumido um compromisso exagerado para todas as histórias ao longo de todo o projeto de desenvolvimento



# HISTÓRIA DE USUÁRIO



- Surgiu no XP, mas é utilizado em várias outras metodologias de desenvolvimento ágil
- É uma técnica de análise de requisitos
- Indique **quem** faz **o quê** e **por quê**
- Simples, limitada
- Pode ser escrita à mão, em um cartão, ficha, post-it

"Como um <papel>, eu quero <meta/desejo> de modo que <benefício>"

"A fim de <receber benefício> como um <papel>, eu quero <meta/desejo>"

"Como um <papel>, eu posso <ação com o sistema> para que <benefício externo>"

"Como um <papel>, eu quero <meta/desejo>"

"Como <quem> <quando> <onde>, eu <o que>, porque <por que>"

# HISTÓRIA DE USUÁRIO

## Busca por clientes

Como um usuário, eu quero procurar por meus clientes pelos seus primeiros e últimos nomes.

## Modificar agendas

Como um usuário não administrativo, eu quero modificar minhas próprias agendas mas não as agendas dos outros usuários.

## Executar testes

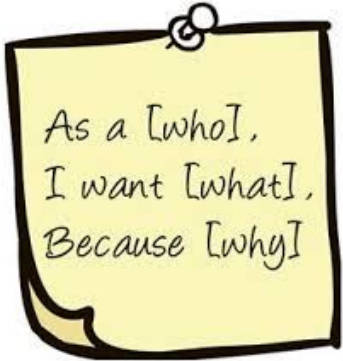
Como um testador de aplicações móveis, eu quero testar meus casos de teste e relatar os resultados para meu gerenciamento.

## Fechar aplicação

Como um usuário fechando a aplicação, eu quero ser perguntado para salvar se eu tiver feito qualquer alteração em meus dados desde a última gravação.

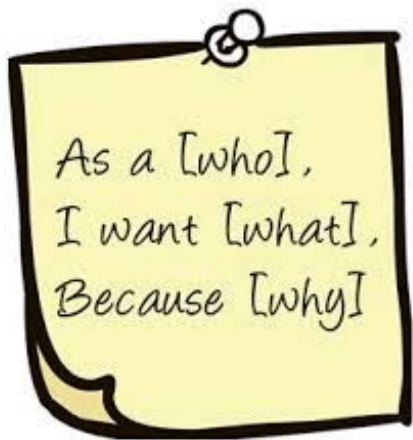
## Iniciar aplicação com última edição

Como um usuário, eu quero iniciar uma aplicação com a última edição.



As a [who],  
I want [what],  
Because [why]

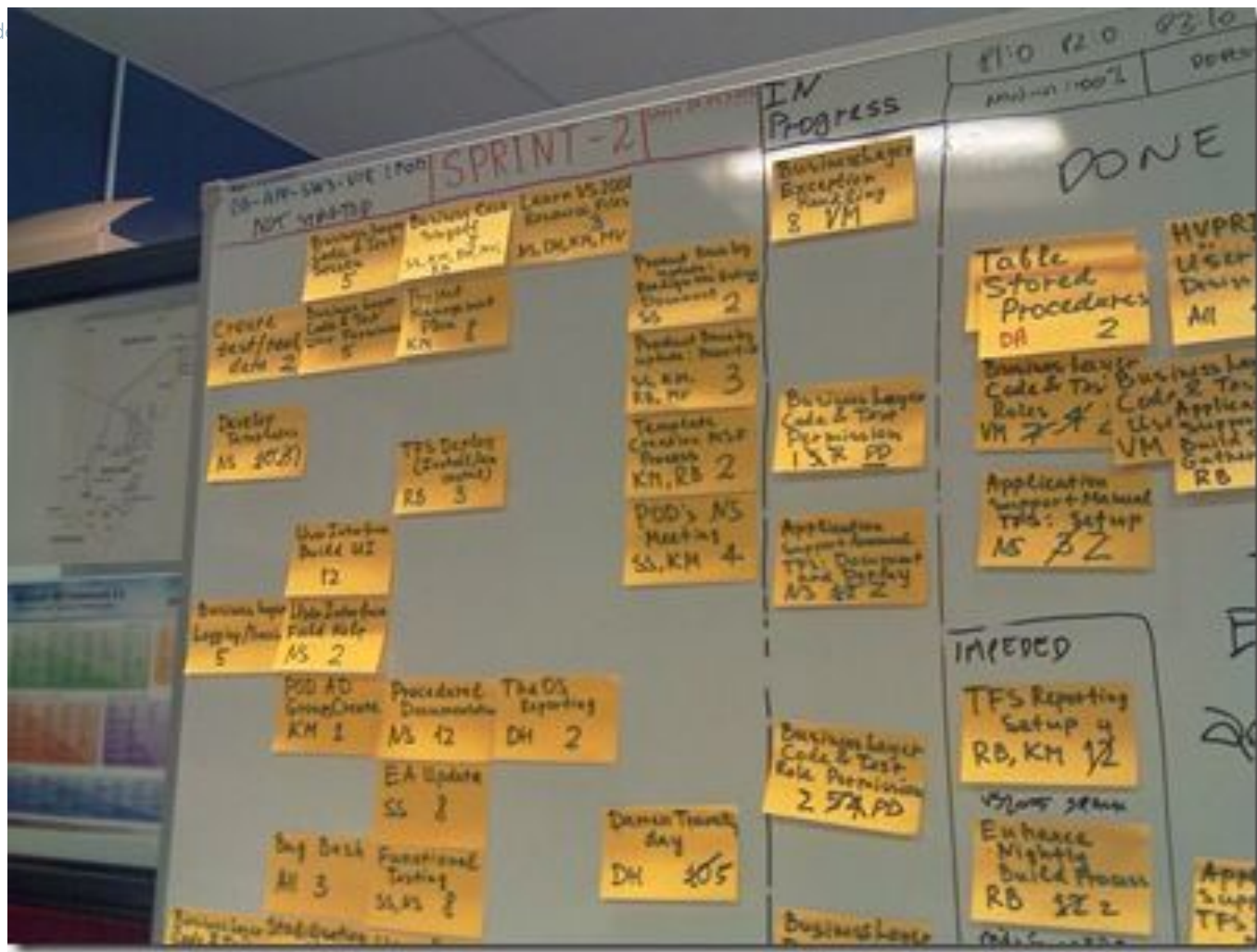
# HISTÓRIA DE USUÁRIO



## Entrar com as despesas

O consultor irá entrar as despesas despesas em um formulário de despesas. O consultor entrará com itens no formulário como tipo de despesa, descrição, quantidade, e quaisquer observações sobre a despesa. A qualquer momento, o consultor pode fazer qualquer uma das seguintes opções:

- (1) Quando o consultor terminar de digitar a despesa, o consultor irá "Enviar". Se a despesa for menor que cinquenta ( $<50$ ), a despesa irá diretamente para o sistema de processos.
- (2) No caso em que o consultor não tenha terminado de entrar com a despesa, o consultor pode querer "Guardar para mais tarde". Os dados inseridos devem, então, ser mostrados em uma lista (fila) para o consultor com o estado de "Incompleta".
- (3) No caso em que o consultor decidir limpar os dados e fechar o formulário, o consultor irá "Cancelar e sair". Os dados introduzidos não serão salvos em parte alguma.



Research

Exploration

Ordering

Checkout

Safety

Understand  
Grocery  
Surplus

Shopping  
related  
Questions

Browse  
by category

Search

Related  
Items

Add to  
basket

Order  
Review

Shipping

Payment

Account

A simple and short  
explanation text  
on why grocery  
surplus is  
good for me.

I'd like to browse  
by top level  
categories.

I'd like to  
add items to  
basket.

I'd like to  
review my  
basket.

I'd like to  
enter my  
address.

I'd like to  
pay using  
PayPal.

I'd like to register  
& login with  
my email  
and password.

I want to see  
if the website  
is trustworthy  
and safe.

A simple and short  
explanation video  
on why grocery  
surplus is  
good for me.

I want a  
knowledgebase  
containing  
FAQs.

I'd like to  
browse by  
second &  
third level  
categories.

I'd like to  
SEARCH  
in product  
titles and tags.

When on PDP  
I'd like to see  
similar items  
from the same  
sub-category.

I'd like to see  
the total weight  
of my basket,  
and the  
remaining  
weight is the padding.

I want to  
live-chat  
with someone to  
get answers to my  
questions.

I'd like a  
full text  
natural  
language search.

I'd like to see  
frequently  
bought  
together items.

I'd like to have  
my basket items  
saved,  
even after I  
close my browser.

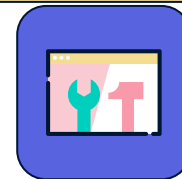
I'd like to  
select  
shipping  
method.

I'd like to  
pay using  
my credit  
card.

I'd like to  
register/login  
with  
Facebook.



# EXTREME PROGRAMMING (XP)



## METODOLOGIA

Orientada a objetos



## ATIVIDADES METODOLÓGICAS

Planejamento, projeto,  
codificação e testes

# XP: PROJETO

## KISS

*Keep It Simple, Stupid!*

Não complique!  
Melhor um projeto simples do que uma representação complexa

## GUIA DE IMPLEMENTAÇÃO

Guia de implementação para uma história à medida que ela é escrita

**Documentação necessária**

## PROJETAR FUNCIONALIDADES EXTRAS

Não é estimulado  
Mesmo o desenvolvedor sabendo que ela será necessária no futuro  
São obstáculos para a simplicidade

**Criando tela para visualizar USUÁRIOS**

Desenvolvedor resolve colocar na tela botões para DELETAR, DESABILITAR (vai ser usado mais pra frente)

## Refatoração

Tela p/ inserir professor  
Tela p/ inserir turmas  
Tela p/ inserir disciplina  
**Tela base com os elementos comuns**

## USO DE CARTÕES CRC

**Classe, Responsabilidade, Colaborador**

Forma de identificar as classes relevantes para os requisitos do sistema  
Analisa as classes relevantes para o incremento

## PROTÓTIPOS

Criados quando temos uma parte de projeto difícil, referente a uma história de usuário

**Solução pontual**

Protótipo é criado e avaliado  
**Objetivo: diminuir o risco**

## REFATORAÇÃO

Aprimorar a estrutura interna de um projeto sem alterar sua funcionalidade ou comportamento

**Organizar o código**

Diminuir a chance de inserir bugs  
Pode ser trabalhoso para projetos grandes



# MODELO CRC

01.

## CLASSE

Nome da classe em questão

02.

## RESPONSABILIDADE

Atributos e operações relevantes para aquela classe

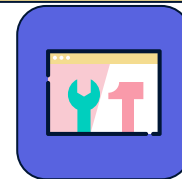
03.

## COLABORADOR

Classes que fornecem a uma classe informações para completar uma responsabilidade

| Classe: Conta Corrente         |                         |
|--------------------------------|-------------------------|
|                                | Classes associadas      |
| Responsabilidade               | Colaboração             |
| Saber o seu saldo              | Cliente                 |
| Saber seu cliente              | Histórico de Transações |
| Saber seu número               |                         |
| Manter histórico de transações |                         |
| Realizar saques e depósitos    |                         |

# EXTREME PROGRAMMING (XP)



## METODOLOGIA

Orientada a objetos



## ATIVIDADES METODOLÓGICAS

Planejamento, projeto,  
codificação e testes

# XP: CODIFICAÇÃO

## TESTES DE UNIDADE

Feito antes da codificação

Testa cada uma das histórias de usuário da versão a ser criada

**Testes estarão falhando**

## INTEGRAÇÃO

O código é integrado ao trabalho dos outros

**Equipe de integração OU Dupla de programadores**

Integração contínua ajuda a encontrar erros mais rapidamente

## PROGRAMAÇÃO EM PARES

Duas pessoas trabalhando juntas em uma mesma estação de trabalho

Devem revezar de função  
Devem trocar de pares  
Códigos sem dono

Duas cabeças pensam melhor do que uma

Mantém os desenvolvedores mais concentrados

Código é revisado assim que é criado (aumenta a qualidade)

**Cada pessoa pode ter um foco diferente**

- Codificação
- Garantia de padrões de codificação
- Garantia de que passará nos testes

É preciso acabar com essa cultura individualista

Aumenta a detecção de bugs

Permite que todos conheçam um pouco mais sobre o sistema

Disseminação de conhecimento

Utilização da "melhor solução"  
Têm-se 2 opções

Diminuição de gambiarras

Aumenta a velocidade do desenvolvimento  
Mesmo sendo 2 pessoas

# WALL STREET JOURNAL (2012)

## ARTIGO SOBRE PROGRAMAÇÃO EM PARES

"A programação em pares está ganhando adeptos em empresas de tecnologia como o Facebook e o Square (a startup de pagamentos móveis). Os defensores da programação em pares falam de forma entusiasmada das suas vantagens, como a facilidade de os pares perceberem erros que seriam custosos se descobertos mais tarde e que há menos possibilidade de se gastar tempo navegando na internet."

"A realidade pode parecer mais como um encontro às cegas fracassado e sem fim. Aborrecimentos entre os parceiros se acumulam rapidamente: desde a falta de higiene pessoal e educação básica, ao compartilhamento de apoios de pés."

"Na Pivotal Labs, uma empresa de desenvolvimento de software comprada pela gigante EMC Corp em março, os 175 engenheiros trabalham em pares o dia todo, todos os dias. Alguns praticam algo conhecido como "pareamento promíscuo", mudando de parceiros diariamente."



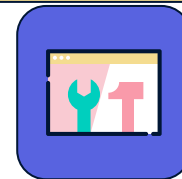
# PAIR PROGRAMMING

**ASSISTA ESSE VÍDEO COM PADRÕES QUE NÃO DEVEM ACONTECER  
NA PROGRAMAÇÃO EM DUPLAS**

<https://www.youtube.com/watch?v=McZ131y0OYU>



# EXTREME PROGRAMMING (XP)



## METODOLOGIA

Orientada a objetos



## ATIVIDADES METODOLÓGICAS

Planejamento, projeto,  
codificação e testes

# XP: TESTES

## TESTE UNITÁRIOS

Devem ser automatizados  
Facilita a execução deles a cada interação no sistema (**testes de regressão**)

## TESTES DE INTEGRAÇÃO E VALIDAÇÃO

Devem ser feitos diariamente  
Alertas no início do desenvolvimento

## TESTES DE ACEITAÇÃO

São obtidos através das histórias do usuário

"Corrigir pequenos problemas em intervalos de poucas horas leva menos tempo do que corrigir problemas enormes próximo ao prazo de entrega"

-- Wells (1999)



03.

EXTREME

PROGRAMMING

Resumo

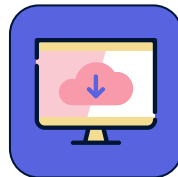


# PRÁTICAS DO EXTREME PROGRAMMING (XP)



## VERSÕES PEQUENAS

Novas versões de no máximo 3 semanas



## JOGO DO PLANEJAMENTO

Definição de custo  
Definição de prioridade  
Definição de cronograma



## TESTE

TDD (*Test-Driven Development*)  
Testes unitários  
Testes de regressão (testes automatizados)  
Testes de integração  
Testes de aceitação



## PROGRAMAÇÃO EM PARES

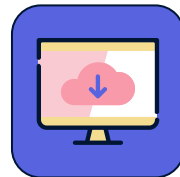
Uma pessoa programa  
Outra pessoa critica e dá sugestões

# PRÁTICAS DO EXTREME PROGRAMMING (XP)



## PROJETO SIMPLES

Permite mudanças facilmente  
Pouca documentação (histórias de usuários, cartões CRC e protótipos)



## REFATORAÇÃO

Melhorar seu código sem mudar as funcionalidades

**Lembrar que não projetamos para frente (mesmo sabendo que às vezes é necessário)**



## PROPRIEDADE COLETIVA

Código sem dono



## INTEGRAÇÃO CONTÍNUA

Diária  
Correção de erros imediatamente (mais fácil de achar e de corrigir)

# PRÁTICAS DO EXTREME PROGRAMMING (XP)



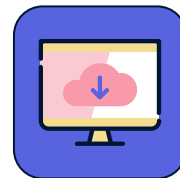
## CLIENTE PRESENTE

Definição de histórias  
Escolha das histórias  
Definição de prioridade  
Testes de aceitação



## PADRÕES DE CODIFICAÇÃO

É importante já que programa-se em pares, e não há dono para os códigos



## SEMANA DE 40H

Manutenção de um ritmo sustentável de trabalho

# EXTREME PROGRAMMING



# DÚVIDAS?

Qualquer dúvida pode entrar em contato:

**Profª. Carla Fernandes Curvelo**  
Email: [carla.fernandes@ufrn.br](mailto:carla.fernandes@ufrn.br)

